

Modül 09: Mühürlü Tipler ve Desen Eşleme

Bu modül, daha önce tanıştığınız üç özelliğin ayrı şeyler olmaktan çıktığı yer. Mühürlü arayüzler durumların neler olduğunu, record'lar her durumun taşıdığı veriyi, kapsayıcı `switch` de bunlarla ne yapılacağını söyler. Derleyici üçünün birbirini tuttuğunu denetler.

Sonuç, diğer dillerin cebirsel veri tipleri dediği şeyin sıradan Java ile yazılmış hâli.

Neler Öğreneceksiniz

- `sealed` ve izin verilen her alt tipin seçmek zorunda olduğu üç niteleyici
- `instanceof` ile desen eşleme ve ortadan kaldırdığı dönüşüm
- Switch desenleri, `when` koşulları ve açık `case null`
- Record desenleri, iç içe parçalama dahil
- Kapsayıcılığın neden asıl kazanç olduğu ve `default` yazmanın size ne kaybettirdiği

Başka Bir Dilden Geliyorsanız

	Başka yerlerde	Java
Kapalı durum kümesi	Rust <code>enum</code> , Kotlin <code>sealed</code> , TS union	<code>sealed interface</code>
Parçalama	Python <code>match</code> , JS destructuring	record desenleri
Koşul	dal içindeki <code>if</code>	<code>when</code> ifadesi
Kapsayıcılık	Rust zorunlu tutar	mühürlü tipler için zorunlu

Rust ya da Kotlin yazdıysanız bu hemen tanıdık gelecek. Eski Java yazdıysanız, veri modelleme biçiminizi en çok değiştirecek özellik bu.

Ders

sealed

```
final      kimse bunu genişletemez
sealed     yalnızca şu isimlendirilmiş tipler genişletebilir
(hiçbiri)  herkes, her yerde, sonsuza dek
```

Modül 06 varsayılan olarak `final` kullanmayı savundu. `sealed`, alt tip *istediğiniz* ama bilinen ve sabit bir küme *istediğiniz* durumun cevabıdır:

```
sealed interface Shape permits Circle, Rectangle, Triangle { }
```

İzin verilen her alt tip kendi niyetini bildirmek zorundadır, `final`, `sealed` ya da `non-sealed` seçerek. Üçünü de yazmamak derleme hatasıdır:

```
error: sealed, non-sealed or final modifiers expected
```

Record'lar örtük olarak `final`'dır, bir record'un neden niteleyiciye ihtiyaç duymadığının sebebi budur. Bu eşleşme bilinçlidir ve modern Java veri modellemesinin çoğunun aldığı biçimdir.

Davetsiz bir tip reddedilir:

```
error: class is not allowed to extend sealed class: Result (as it is not listed in its
'permits' clause)
record Maybe(String hint) implements Result {
^
```

Durum kümesi tasarımın parçasıysa mühürleyin. Bir ödeme karttır, havaledir ya da hediye çekidir. Bir ayrıştırma ya başarılı olmuştur ya olmamıştır. Bunlarda "biri sonradan başka bir tane ekleyebilir" istediğiniz bir genişletme noktası değil, yakalanmasını istediğiniz bir risktir.

Eklenti arayüzünü mühürlemeyin. Üçüncü tarafların uygulaması gereken bir şeyi mühürlemek, kimsenin genişletemeyeceği bir kütüphane yayımlamanın yoludur.

Desen eşleme

`case String s` tipi test eder, dönüştürür ve sonucu adlandırır, hepsi bir arada. Aynı mantığı iki kez yazan `PatternMatching.java` dosyasından:

```
// modern
case String s when s.isEmpty() -> "an empty string";
case String s -> "text of length " + s.length();

// old
if (o instanceof String) {
    String s = (String) o;
    return s.isEmpty() ? "an empty string" : "text of length " + s.length();
}
```

Eski biçim tip adını iki kez yazar ve ayrı bir dönüşüm yapar. O dönüşüm hataların yaşadığı yerd: hiçbir şey sizi `if (o instanceof String) { Integer i = (Integer) o; }` yazmaktan alıkoymaz, ki bu derlenir ve fırlatır.

Bilinmeye değer üç ayrıntı:

- **Koşullar** `when` kullanır, ve koşullu durumlar aynı tip için koşulsuz olandan önce gelmelidir. İlk eşleşme kazanır, dolayısıyla başa konan koşulsuz bir `case String s` her şeyi yutar.
- **case null** açıktır. Onsuz, null üzerinde switch yapmak uyumluluk için korunmuş önceki davranışla `NullPointerException` fırlatır.
- **instanceof** desenleri **switch dışında da çalışır**, ki onları en çok orada kullanacaksınız:

```
if (o instanceof String s && s.length() > 5) {  
    return s.substring(0, 5) + "...";  
}
```

Kapsam kuralı görüldüğünden zekidir. Erken bir dönüşten sonra değişken metodun geri kalanında kapsamdadır:

```
if (!(o instanceof String s)) {  
    return -1;  
}  
return s.length(); // s is in scope here
```

Record desenleri

Desenin kendisinde parçalama. `RecordPatterns.java` dosyasından:

```
case Circle(Point(var x, var y), var r) when x == 0 && y == 0 ->  
    "circle of radius " + r + " at the origin";  
  
case Rectangle(Point(var x1, var y1), Point(var x2, var y2))  
    when (x2 - x1) == (y2 - y1) ->  
    "a square of side " + (x2 - x1);
```

Gerçek çıktı:

```
circle of radius 5.0 at the origin  
a square of side 4  
a 6 by 3 rectangle
```

Kare tespiti için `Rectangle` üzerinde hiçbir metoda gerek olmadı. Desen tipi eşleştirdi, iç içe iki noktayı çıkardı, dört değeri bağladı ve karşılaştırdı, hepsi tek bir case etiketinde.

Kapsayıcılık asıl kazanç

Mühürlü tip üzerindeki bir switch `default` istemez, çünkü derleyici eksiksiz olduğunu kanıtlayabilir. Bir durumu dışarıda bırakın ve `NotExhaustive.java` gerçekleştir:

```
error: the switch expression does not cover all possible input values  
    return switch (payment) {  
        ^  
missing patterns:  
    Voucher _
```

Eksik tipi ismen söylüyor. `Voucher _` içindeki alt çizgi isimsiz bir desendir: derleyici size `Voucher` için bir dal gerektiğini ve değeri yok sayan bir dalın yeteceğini söylüyor.

Mühürlemenin tüm gerekçesi bu. `permits` listesine bir sabit ekleyin ve tüm kod tabanındaki her eksik switch kendini gösteren bir derleme hatasına dönüşür. Onun yerine `default -> 0` yazmak, ücretsiz bir

hediye çekini uyarısız gönderirdi.

Modül 03 bu argümanı enum'lar için yapmıştı. Mühürlü tipler onu, verisi olan kapalı herhangi bir biçim kümesine genişletiyor.

Çalıştırın

```
java modules/09-sealed-and-pattern-matching/examples/SealedTypes.java
java modules/09-sealed-and-pattern-matching/examples/PatternMatching.java
java modules/09-sealed-and-pattern-matching/examples/RecordPatterns.java

# İkisi de bilerek başarısız olur. Hangi tipi isimlendirdiklerini okuyun.
java modules/09-sealed-and-pattern-matching/examples/NotExhaustive.java
java modules/09-sealed-and-pattern-matching/examples/NotPermitted.java

./scripts/verify-examples.sh modules/09-sealed-and-pattern-matching
```

Sık Yapılan Hatalar

Mühürlü tip üzerindeki switch'e default eklemek. Gelecekteki her derleme hatasını sessiz bir yanlış cevaba çevirdiniz. Bu modüldeki en önemli alışkanlık budur.

Koşulsuz durumu başa koymak. `case String s when ...` öncesinde `case String s` yazmak koşullu dalı erişilemez kılar, ve derleyici bunu söyler.

case null unutmak. Listeremediğiniz sürece null üzerinde switch fırlatır. Null mümkünse görünür biçimde ele alın.

Üçüncü tarafların uygulaması gereken bir arayüzü mühürlemek. Mühürleme sahip olduğunuz kapalı kümeler içindir, genişletme noktaları için değil.

İzin verilen bir alt tipi niteleyicisiz bildirmek. Her biri `final`, `sealed` ya da `non-sealed` demek zorunda. Record'lar zaten `final` diyor.

Mühürlü hiyerarşinin uyduğu yerde derin kalıtıma uzanmak. Alt tipler davranıştan çok veride ayrışıyor, record'lardan oluşan mühürlü bir arayüz artı switch, geçersiz kılmalı soyut sınıftan genellikle daha nettir.

Özet Çıkarımlar

- sealed alt tiplerin eksiksiz kümesini isimlendirir**, ve her biri `final`, `sealed` ya da `non-sealed` bildirmek zorundadır.
- Desenler tek adımda test eder, dönüştürür ve bağlar**, eskiden yanlış giden dönüşümü ortadan kaldırarak.

- Koşullar `when` kullanır ve aynı tip için koşulsuz durumdan önce gelmelidir.
- `case null` açıktır; aksi hâlde null fırlatır.
- Record desenleri parçalar, ve iç içe girer, veri kadar derine.
- Mühürlü tip üzerinde `default` yazmayın. Eksik durum derleme hatası, satın aldığınız özelliktir.
- Mühürlü arayüz + record'lar + kapsayıcı switch, modern Java'nın kapalı bir veri biçimi kümesini modelleme yoludur.

Ödev

[homework/README.md](#)

Bir ifade değerlendiricisini mühürlü hiyerarşi olarak modelleyin, sonra derleyicinin unuttuğunuz durumu yakaladığını kanıtlayın. Referans çözüm [solutions/09-sealed-and-pattern-matching/](#) altında.